Instituto Politécnico de Beja
Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática
Programação Orientada por Objetos

Smart Food Basket - ODS Edition

Trabalho Prático de Avaliação

Beja, 8 de abril de 2026

Ano Letivo 2025/2026.

Este enunciado pode sofrer pequenas alterações até à data limite de entrega.
Deve estar atento ao Teams

A gente não quer só comida
A gente quer comida, diversão e arte
A gente não quer só comida
A gente quer saída para qualquer parte
– Comida, Titãs

`https://www.youtube.com/watch?v=hD36s-LiKlg`

`https://www.youtube.com/watch?v=hOyt4cwjVns`

1 Introdução

Este enunciado é algo extenso. Tal significa que deve ser lido mais do que uma vez e com muita atenção. O que é pedido está classificado em três tipos de **REQUISITOS**:

1. **ESSENCIAIS**
2. **NÃO ESSENCIAIS**
3. **QUE IMPLICAM PENALIZAÇÕES.**

Os essenciais permitem obter 10 valores, mas apenas desde que não estejam presentes motivos para penalizações. Os requisitos não essenciais permitem obter até 17 valores. Os 18, 19 e 20 valores ficam para quem fez tudo o que é pedido e implementou **EXTRAS** com dificuldade superior.

- Só pode ter mais de 10 valores se cumprir totalmente os requisitos essenciais e não tiver penalizações;
- Só pode ter mais de 17 valores se cumprir totalmente os requisitos essenciais e não essenciais e não tiver penalizações.

2 A Aplicação

Neste trabalho prático, pretende-se desenvolver uma aplicação com interface gráfica inspirada em jogos de *tiles* focada nos Objetivos de Desenvolvimento Sustentável (ODS), mais especificamente o ODS 2 (Fome Zero), o ODS 3 (Saúde), e o ODS 12 (Consumo e Produção Responsáveis). O utilizador deverá construir um *Smart Food Basket* (Cabaz Alimentar Inteligente).

Interface Baseada em Peças Cada alimento disponível no "supermercado"é representado por um *tile* (peça) na interface gráfica. Esta peça deve ser colorida de acordo com o seu grupo da Roda dos Alimentos: `GREEN` (Hortícolas/-Frutas), `YELLOW` (Cereais), `BLUE` (Laticínios), `BROWN` (Leguminosas), `RED` (Carnes/Peixe) ou `ORANGE` (Gorduras).

Herança e Polimorfismo Para introduzir complexidade e realismo, os alimentos dividem-se em dois tipos: **Embalados** (`PackagedFood`, com preço unitário fixo) e a **Granel** (`BulkFood`, cujo preço e impacto ambiental dependem do peso exato escolhido pelo utilizador).

Métricas ODS A aplicação atua como um "treinador de sustentabilidade", calculando em tempo real o custo total, as calorias totais e verificando se o utilizador atingiu metas saudáveis (ex: comer pelo menos 400g de vegetais por dia).

Persistência de Dados Todos os produtos disponíveis são lidos de um ficheiro `foods.csv` no arranque da aplicação, garantindo que o programa funciona com qualquer lista de alimentos que lhe seja fornecida. Este ficheiro deve ser feito e deve haver pelo menos 3 ficheiros com conteúdos e dimensões diferentes. Tal permitirá um teste mais completo.

É obrigatório basear o código no *template* `JavaFX-PO2-2023-2024` e respeitar a separação `ui` e `model` e a comunicação via interface `View`. Caso não respeite a arquitetura exigida, a nota será de zero valores. No entanto, pode sempre escrever mais classes e/ou mais métodos do que os solicitados. Deve preferir mais classes e métodos com poucas linhas em lugar de menos classes e métodos com muitas linhas.

3 Requisitos Essenciais

O incumprimento de qualquer destes requisitos implica uma classificação negativa.

Req. 1 - Hierarquia de Classes (1,0 valores) Deve implementar a seguinte arquitetura:

1. **Enum `FoodGroup`:** Define os grupos alimentares e as cores associadas a usar na interface.
2. **Classe abstrata `FoodItem`:** Deve definir o método abstrato `double getPrice()` e `double getCalories()`. Pode conter atributos.
3. **Subclasse `PackagedFood`:** Herda de `FoodItem`. Representa itens vendidos à unidade (ex: 1 pacote de arroz). Adiciona o atributo `String`

`packagingMaterial` (ex: "Plastico", "Cartao"). Os métodos `getPrice()` e `getCalories()` devolvem os valores base sem alterações.

4. **Subclasse `BulkFood`:** Herda de `FoodItem`. Representa itens vendidos ao peso (ex: maçãs). Adiciona o atributo `double selectedWeight` (peso em gramas escolhido pelo utilizador). O método `getPrice()` deve calcular o valor proporcional (ex: se o preço base for 2.00€/kg, e o utilizador selecionar 500g, o método devolve 1.00€).
5. **Classe `BasketModel`:** Gere o estado da aplicação. Contém a lista polimórfica de itens do cabaz (`List<FoodItem> items`) e os métodos para calcular os totais do cabaz (iterando a lista e chamando `getPrice()`).

Req. 2 - Testes Unitários (4,0 valores) A classe `BasketModel` e o cálculo dos preços devem ser testados rigorosamente usando JUnit:

`testPackagedPrice()`: Garante que o método `getPrice()` de um `PackagedFood` devolve exatamente o preço unitário definido.

`testBulkPriceCalculation()`: Garante que o método `getPrice()` de um `BulkFood` efetua a regra de três simples corretamente com base no peso fornecido.

`testTotalBasketCalculation()`: Adiciona um misto de alimentos (Embalados e a Granel) ao `BasketModel` e verifica se a soma total está correta, comprovando que o polimorfismo está a funcionar.

`testSDGGoalCondition()`: Testa a lógica da meta das 400g. Cria um cabaz com mais de 400g de alimentos do grupo `GREEN` e verifica se o método que valida a meta devolve `true`. Testa também com menos de 400g esperando `false`.

Req. 3 - Gestão de Dados CSV (1,5 valores) A classe `BasketModel` (ou um `FoodFactory`) deve conter um método para ler o ficheiro `foods.csv`. O ficheiro tem o formato `Tipo,Nome,Grupo,PrecoBase,CaloriasBase,InfoExtra`. O código deve ler o primeiro campo (`Tipo`) para decidir dinamicamente se instancia um `PackagedFood` ou um `BulkFood`. O código deve ler o ficheiro linha a linha. Apenas pode utilizar as classes de leitura de ficheiros e de processamento de strings que estão no JDK. Linhas com erros devem ser ignoradas e, após a leitura do ficheiro, deve surgir uma caixa de diálogo com todo o texto de todas as linhas que foram ignoradas.

Req. 4 - Interface Gráfica e Interação (1,5 valores) A janela principal deve estar dividida em duas áreas:

1. **A Grelha (*Grid*):** Um `GridPane` que cria um botão/painel colorido para cada alimento lido do CSV. O botão deve exibir o nome e o preço base.
2. **Interação Polimórfica:** Ao clicar num *tile* de um `PackagedFood`, o item é adicionado logo ao cabaz. Ao clicar num `BulkFood`, a *ui* tem de exibir uma caixa de diálogo (`TextInputDialog`) a perguntar: "Quantas gramas deseja adicionar?". Só depois disso a nova instância (com o peso atualizado) é inserida no `BasketModel`.
3. **Dashboard:** Um painel lateral que se atualiza automaticamente sempre que a lista de itens do modelo sofre alterações, apresentando o Custo Total, Calorias Totais e a Lista de Nomes dos itens comprados.

Req. 5 - Exportação do Recibo (1,0 valores) A interface deve conter um botão "Finalizar Compra". Quando clicado, a aplicação itera sobre os elementos da lista do cabaz e escreve num ficheiro `receipt_[AAAAMMDD].txt` o nome de cada alimento, a quantidade/peso escolhido, o valor pago por esse item, e no final o Grande Total. Deve também imprimir uma mensagem ("Meta ODS 3 Cumprida!"ou "Meta ODS 3 Falhada!").

Req. 6 - Visualização de Estados (1,0 valores) Diferencie visualmente, no próprio *tile* da interface gráfica, se um alimento é "Embalado"ou "A Granel"(pode usar um pequeno ícone de uma caixa vs uma balança, ou texto indicativo como "€/un."vs "€/kg").

4 Requisitos Não Essenciais

Req. 7 - Emblemas ODS Dinâmicos (1,0 valores) Coloque na interface três emblemas visuais (ODS 2, 3 e 12) que começam opacos (cinzentos). O `BasketModel` deve disparar eventos para a *View* para "acender"os emblemas com as suas cores originais quando as regras são cumpridas:

- **ODS 3 (Saúde):** Atingiu as 400g no grupo verde.
- **ODS 12 (Sustentabilidade):** Mais de 50% dos itens no cabaz são a Granel (`BulkFood`) E nenhum `PackagedFood` tem "Plastico"no material.
- **ODS 2 (Fome Zero/Nutrição):** O cabaz possui pelo menos um item de 4 `FoodGroup` diferentes (diversidade alimentar).

Req. 8 - Filtros e Ordenação (1,5 valores) Adicione botões acima da grelha que permitam filtrar a vista: "Mostrar Todos", "Só Fruta/Vegetais", "Só Proteínas". Adicione um `ComboBox` que permita ordenar os *tiles* visíveis por "Preço Menor a Maior"ou "Mais calóricos primeiro".

Req. 9 - Geração Aleatória Inteligente (1,5 valores) Crie um botão "Cabaz Saudável Automático". O algoritmo deve escolher itens aleatórios do CSV, adicionar pesos aleatórios (no caso do granel), e verificar iterativamente as métricas até gerar um cabaz que acenda, no mínimo, dois dos emblemas ODS referidos no Req 7, sem ultrapassar os 20€.

Req. 10 - Sistema de *Undo/Redo* (1,0 valores) O utilizador enganou-se a inserir as gramas de um produto. Deve existir um atalho (ex: CTRL+Z) ou botão no menu que permita remover o último item inserido e atualizar todos os valores do Dashboard de imediato. O CTRL+Y deve repor.

Req. 11 - Sonorização e Animação (1,0 valores) Adicione um som agradável (*ding*) quando uma meta ODS é atingida. Adicione uma animação JavaFX simples (*FadeTransition* ou *ScaleTransition*) aos *tiles* quando estes são clicados ou filtrados.

Req. 12 - Replay de Histórico (1,0 valores) Adicione um menu "Importar Recibo". A aplicação lê um ficheiro `receipt.txt` gerado anteriormente pelo Req. 5, limpa o cabaz atual, e adiciona os itens um a um com um atraso de meio segundo entre eles (*delay*), reconstruindo o cabaz visualmente perante o utilizador.

5 Requisitos que podem implicar penalizações (RPB)

A falha em cumprir as regras de engenharia de software anula o esforço tido nos requisitos funcionais.

RPB 1 - Separação de Camadas – 0 a -5 valores A classe da interface (*View/Controller*) não pode conter regras de negócio (como fazer `if (peso > 400)`). A

interface apenas envia o clique para o *Model*, que faz as contas e notifica a *View* do novo valor a apresentar.

RPB 2 - Polimorfismo É proibido utilizar `instanceof`, `getClass()`, ou mecanismos de *casting* manual para descobrir se um alimento é *Bulk* ou *Packaged* na altura de somar os preços. O `BasketModel` deve apenas pedir `item.getPrice()` e confiar na ligação dinâmica.

RPB 3 - Identificadores Todo o código (nomes de variáveis, métodos, classes) deve estar redigido em **inglês**. O texto apresentado ao utilizador final na *UI* pode estar em português.

RPB 4 - Regras de Estilo Os métodos devem ser curtos (máximo 20 pontos e vírgulas). É obrigatório o uso do `this.` ao referenciar atributos da classe. As constantes devem usar **`final`**. O Javadoc é obrigatório antes da definição de cada método público. **Não pode colocar comentários dentro dos métodos.**

6 Regras de Utilização de Inteligência Artificial Generativa

Nesta Unidade Curricular, o uso de ferramentas de Inteligência Artificial Generativa (como ChatGPT, GitHub Copilot, Gemini, Claude, etc.) é **permitido, mas estritamente regulamentado**. O objetivo é que sejam utilizadas como auxiliares de aprendizagem e de produtividade, e não como substitutos do raciocínio algorítmico e arquitetural.

Se utilizarem IA no desenvolvimento deste projeto, têm obrigatoriamente de cumprir os seguintes pontos:

1. O Diário de IA (Ficheiro Obrigatório) Se utilizou IA para gerar, refatorizar ou corrigir código, o ficheiro `.zip` final tem de incluir um ficheiro de texto chamado `relatorio_ia.txt` que respeite a minuta na secção seguinte. O não envio deste ficheiro (caso se detete uso de IA) implicará zero valores na nota final e a consequente reprovação da unidade curricular, em época normal e de recurso.

2. Adaptação às Regras da UC O facto de a IA ter gerado algo não será aceite como justificação para o incumprimento de regras.

3. Defesa Oral Durante a discussão do projeto, será questionado sobre a lógica, a arquitetura e os detalhes de implementação do código entregue. Qualquer porção de código que o aluno não consiga explicar e justificar detalhadamente implicará zero valores na nota final e a consequente reprovação da unidade curricular, em época normal e de recurso, independentemente da origem do código.

7 Minuta para relatório de utilização de IA

```
=================================================================
Relatório de utilização de IA
=================================================================
```

```
Nome(s): [O nome e número dos autores do trabalho entregue]
Ferramentas Utilizadas: [ex: ChatGPT 4, GitHub Copilot]

1. RESUMO DA UTILIZAÇÃO
(Descreva em 2 ou 3 frases para que partes do trabalho usou mais a IA.
Ex: "Utilizámos o Copilot para autocompletar código repetitivo na interface
gráfica e o ChatGPT para me explicar como ler ficheiros CSV em Java.")

2. EXEMPLO DE PROMPT E CORREÇÃO
(Coloque aqui UM exemplo representativo de um problema que a IA não
resolveu perfeitamente à primeira e como teve de intervir).

* O que pedi à IA (Prompt):
"Escreve um método para calcular o preço total da lista de compras."

* O que a IA gerou (Resumo do problema):
"A IA gerou um ciclo for com if/else e usou instanceof para ver se a
comida era a granel ou embalada."

* Como eu corrigi/adaptei:
"Como o enunciado proíbe o uso de instanceof, tive de alterar a lista
para guardar o tipo abstrato FoodItem e chamei o método getPrice()
para usar polimorfismo dinâmico."

3. DECLARAÇÃO DE RESPONSABILIDADE
Nós os alunos que submeteram este trabalho declaram que compreendem
totalmente o código presente neste projeto e que são capazes de
reproduzir a sua lógica e justificar as suas decisões arquiteturais
de forma independente e sem qualquer tipo de auxílio.
================================================================
```

8 Entrega

O trabalho deve ser entregue no Moodle até às **23:59 de 25 de maio de 2026** num ficheiro `.zip`, com o nome no formato `Numero-PrimeiroUltimo-Numero-PrimeiroUltimo-TP-PO2-2025-2026.zip`, e contendo os seguintes dois ficheiros:

1. Um ficheiro de texto (.txt) O relatório de utilização de IA de acordo com a minuta na Secção 7;
2. Um ficheiro zip como projeto IntelliJ depois de removida a pasta `target`.

Todos os autores do trabalho devem entregar o mesmo ficheiro zip no moodle.

Note que a qualquer momento pode ser publicada uma versão revista deste enunciado para clarificação ou correção do mesmo. Deve estar atento ao Teams e consultar regularmente o canal.

Os docentes da UC